Symbol Table

2025-2026

LT

3rd Class

Symbol Table

- The data structure that is created and maintained by the compilers for information storing regarding the occurrence of various entities like names of variables, functions, objects, classes
- Symbol table is used by both the analysis and the synthesis parts of a compiler

Symbol Table

- A symbol table may serve the following purposes depending upon the language in hand:
  - To store the names of all entities in a structured form at one place
  - To verify if a variable has been declared
  - To implement type checking, by verifying assignments and expressions in the source code are semantically correct
  - To determine the scope of a name (scope resolution)

Information Stored in Symbol Table

- The following possible information about identifiers are stored in symbol table
  - The name (as a string)
  - Attribute: Reserved word, Variable name, Type name, Procedure name, Constant name
  - The data type
  - The block level
  - Its scope (global, local, or parameter)
  - Its offset from the base pointer (for local variables and parameters only)

Implementation

- Symbol table can be implemented as
  - Unordered List
  - Linear (sorted or unsorted) list
  - Binary Search Tree
  - Hash table
- Among all, symbol tables are mostly implemented as hash tables, where the source code symbol itself is treated as a key for the hash function and the return value is the information about the symbol.

Entry Format

- A symbol table maintains an entry for each name in the following format:

  <symbol name, type, attribute>

- For example, if a symbol table has to store information about the following variable declaration:

  static int interest;

- then it should store the entry such as:

  <interest, int, static>

Operations

- A symbol table, either linear or hash, should provide the following operations.
  - insert()
    - This operation is more frequently used by analysis phase where tokens are identified and names are stored in the table.
    - This operation is used to add information in the symbol table about unique names occurring in the source code.
    - The format or structure in which the names are stored depends upon the compiler in hand.

Operations

- An attribute for a symbol in the source code is the information associated with that symbol.
  - This information contains the value, state, scope, and type about the symbol.
- The insert() function takes the symbol and its attributes as arguments and stores the information in the symbol table.
- For example:

  int a;

  should be processed by the compiler as:

  insert(a, int);

Operations

- lookup()
  - lookup() operation is used to search a name in the symbol table to determine:
    - if the symbol exists in the table.
    - if it is declared before it is being used.
    - if the name is used in the scope.
    - if the symbol is initialized.
    - if the symbol declared multiple times.
- The basic format should match the following:

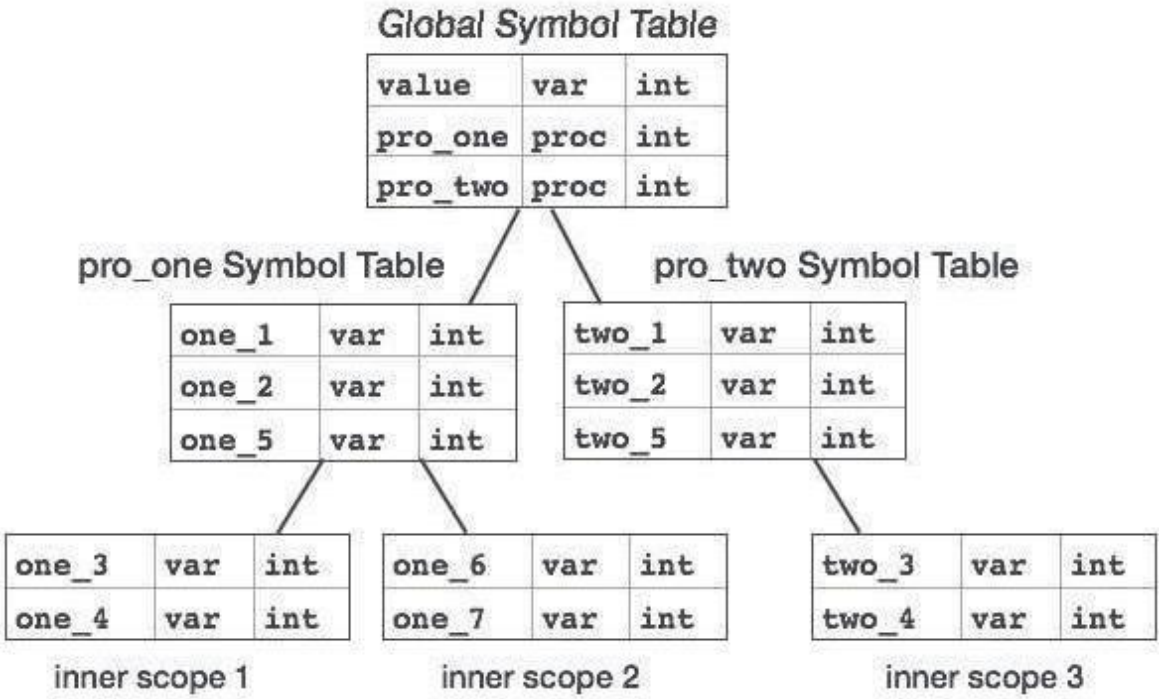
  lookup(symbol)

Operations

- This method returns 0 (zero) if the symbol does not exist in the symbol table. If the symbol exists in the symbol table, it returns its attributes stored in the table.

Scope Management

- Symbol tables are arranged in hierarchical structure as shown in the example below:




```
. . .
int value=10;

void pro_one()
   {
   int one_1;
   int one_2;

      {                  \
      int one_3;          |_  inner scope 1
      int one_4;          |
      }                  /

   int one_5;

      {                  \
      int one_6;          |_  inner scope 2
      int one_7;          |
      }                  /
   }

void pro_two()
   {
   int two_1;
   int two_2;

      {                  \
      int two_3;          |_  inner scope 3
      int two_4;          |
      }                  /

   int two_5;
   }
. . .
```

Symbol table example

```
class Foo {
  int value;
  int test() {
  int b = 3;
    return value + b;

  }
  void setValue(int c) {
    value = c;
    { int d = c;
      c = c + d;
      value = c;

    }
  }
}

class Bar {
  int value;
  void setValue(int c) {
            value = c;
  }
}
```

scope of b

block1

scope of d

scope of c

scope of value

scope of c

scope of value

Symbol table example cont.

⋮

(Foo)

Symbol	Kind	Type	Properties
value	field	int	…
test	method	-> int	
setValue	method	int -> void	

(Test)

Symbol	Kind	Type	Properties
b	var	int	…

(setValue)

Symbol	Kind	Type	Properties
c	var	int	…

(block1)

Symbol	Kind	Type	Properties
d	var	int	…